

Exercícios — Aula 19 — Backtracking com restrições

Técnicas de Programação - 2026/02

Última atualização: July 26, 2026

Instruções

- Antes de podar, escreva a hipótese que torna a poda correta.
- Diferencie poda de heurística: poda preserva corretude; heurística pode falhar.
- Em simulações, conte chamadas visitadas e ramos cortados.

Exercício 1 — Soma-alvo com poda

Simule o algoritmo para $v = [4, 7, 2, 9]$ e $alvo = 11$.

Algorithm 1: ExisteSoma

Result: Executa EXISTE-SOMA.

Input : array v , int $alvo$

Output: boolean

return *BUSCAR*(v , 0, 0, $alvo$)

Algorithm 2: Buscar

Result: Executa BUSCAR.

Input : array v , int i , int $soma$, int $alvo$

Output: boolean

```
if  $soma = alvo$  then
    return true
end
if  $soma > alvo$  then
    return false
end
if  $i = TAMANHO(v)$  then
    return false
end
if BUSCAR( $v$ ,  $i + 1$ ,  $soma + v[i]$ ,  $alvo$ ) = true then
    return true
end
return BUSCAR( $v$ ,  $i + 1$ ,  $soma$ ,  $alvo$ )
```

Preencha a árvore de chamadas até o algoritmo retornar.

```
BUSCAR(i=0, soma=0)
|-- inclui 4 -> BUSCAR(i=1, soma=4)
|   |-- inclui 7 -> ...
|   |-- não inclui 7 -> ...
|-- não inclui 4 -> ...
```

Marque cada ramo como:

- encontrou solução;
- podado por $soma > alvo$;
- terminou sem solução.

Qual subconjunto soma 11?

Exercício 2 — Quando a poda é inválida

A poda `soma > alvo` depende de todos os valores restantes serem positivos.

Considere `v = [8, -3, 2]` e `alvo = 5`.

1. Se começamos incluindo 8, a soma parcial passa do alvo?
2. Ainda existe forma de voltar para 5 usando um número negativo?
3. Qual solução existe?
4. Por que podar quando `soma > alvo` seria incorreto nesse caso?

Agora crie outro exemplo com números negativos em que essa poda descartaria uma solução válida.

Exercício 3 — Mochila 0/1 em três itens

Temos capacidade $C = 5$.

item	peso	valor
0	3	7
1	4	9
2	2	4

Simule a árvore de decisões incluir/não incluir.

Estado da chamada:

(i, peso_atual, valor_atual, escolhidos)

Use a poda:

Algorithm 3: PodaPorCapacidade

```
if peso_atual > C then
  | return
end
```

Preencha:

folha ou poda	escolhidos	peso	valor	válida?
---------------	------------	------	-------	---------

Qual é a melhor solução válida?

Exercício 4 — Atualização da melhor solução

Complete o pseudocódigo da mochila para atualizar a melhor solução quando chega ao caso base.

Algorithm 4: Mochila

Result: Executa MOCHILA.

Input : arrays pesos e values, int capacidade

Output: melhor solução

```

melhor_valor ← 0
melhor_escolha ← ARRAY-DE-FALSOS(TAMANHO(pesos))
escolha_atual ← ARRAY-DE-FALSOS(TAMANHO(pesos))

BUSCAR-MOCHILA(0, 0, 0)
return (melhor_valor, melhor_escolha)

```

Algorithm 5: BuscarMochila

Result: Executa BUSCAR-MOCHILA.

Input : int *i*, int *peso_atual*, int *valor_atual*

Output: none

```

if peso_atual > capacidade then
    return
end
if i = TAMANHO(pesos) then
    if _____ then
        melhor_valor ← _____
        melhor_escolha ← _____
    end
    return
end
escolha_atual[i] ← true
BUSCAR-MOCHILA(i + 1, peso_atual + pesos[i], valor_atual + valores[i])
escolha_atual[i] ← false
BUSCAR-MOCHILA(i + 1, peso_atual, valor_atual)

```

Responda:

1. Por que *melhor_escolha* precisa receber uma cópia?
2. Por que a melhor solução só é atualizada se a solução atual é válida?
3. O que deve acontecer antes de iniciar uma nova execução do algoritmo?

Exercício 5 — Heurística versus busca completa

Considere a mochila com capacidade 10.

item	peso	valor
0	9	19
1	5	10
2	5	10
3	4	7

Compare duas estratégias.

Heurística: escolher primeiro o item de maior valor que ainda cabe.

Busca completa: explorar incluir/não incluir todos os itens, respeitando a capacidade.

Responda:

1. Qual solução a heurística encontra?
2. Qual é o valor dessa solução?
3. Qual solução ótima a busca completa encontra?
4. Qual é o valor ótimo?
5. Por que a heurística não é garantia de ótimo?

Exercício 6 — Medindo chamadas com e sem poda

Compare duas versões de soma-alvo para $v = [3, 4, 5, 6]$ e $\text{alvo} = 9$.

Versão A: explora todos os ramos até $i = \text{TAMANHO}(v)$.

Versão B: corta quando $\text{soma} > \text{alvo}$.

Preencha a tabela.

versão	chamadas visitadas	ramos podados	encontrou solução?
sem poda			
com poda $\text{soma} > \text{alvo}$			

Depois responda:

1. A poda muda o conjunto de soluções corretas para valores positivos?
2. A poda garante que o pior caso deixa de ser exponencial?
3. Em que tipo de entrada essa poda tende a ajudar mais?
4. Qual hipótese precisa aparecer na justificativa?

Créditos e reaproveitamento

Exercícios adaptados de heurísticas da mochila, mochila por backtracking, estado parcial, incluir/não incluir e atualização de melhor solução dos handouts antigos devem indicar:

Adaptado de material de Igor Montagner para a disciplina Técnicas de Programação.

Definições conceituais externas opcionais: Knapsack, https://en.wikipedia.org/wiki/Knapsack_problem, e Subset Sum, https://en.wikipedia.org/wiki/Subset_sum_problem.